Apartment Commute Optimizer

Report 1

CS5800 Spring 2026 Group Project

Xujing Hui (#002339163)

Fanchao Yu (#001601606)

Jiaxin Liu (#003169274)

1. Executive Summary

Our project is on track. We have completed the problem formulation, designed the full graph model using real San Jose locations, implemented all core algorithms (Dijkstra with min-heap and Bellman-Ford for comparison), and produced a working end-to-end prototype that outputs ranked apartment results. The GitHub repository contains all source code, test data, and 15 passing test cases across three test files. Our primary remaining work is expanding the dataset and refining the report and presentation materials.

2. Problem Statement

2.1 Research Question

Among a set of candidate apartments in San Jose, which feasible apartment minimizes total weekly commute time to a fixed set of destinations, subject to rent and transit-accessibility constraints?

2.2 Inputs

5 candidate apartments along the VTA Blue Line, each with a known location (latitude, longitude) and monthly rent ($1,350 to $2,200).

3 fixed destinations: NEU Campus (5 visits/week), Walmart Neighborhood Market (2 visits/week), Planet Fitness (3 visits/week).

8 VTA Light Rail stations (Blue Line segment from St. James to Branham) as intermediate transit nodes, with 3-minute intervals between adjacent stations.

Candidate Apartments

ID	Area	Nearest Station	Rent	Latitude	Longitude
A1	Downtown (San Carlos St)	San Carlos	$2,200	37.3333	-121.8907
A2	Japantown (N 6th St)	Japantown/Ayer	$1,800	37.3490	-121.8863
A3	Tamien Area	Tamien	$1,500	37.3120	-121.8830
A4	Capitol Area	Capitol	$1,400	37.2882	-121.8568
A5	Branham Area	Branham	$1,350	37.2720	-121.8445

Fixed Destinations

ID	Place	Address	Visits/Week	Nearest Station
D1	NEU Campus	4 N 2nd St, SJ 95113	5	St. James
D2	Walmart Neighborhood Market	777 Story Rd, SJ 95122	2	Curtner

D3	Planet Fitness	Blossom Hill Rd, SJ 95123	3	Branham

VTA Blue Line Stations (North to South)

ID	Station Name	Latitude	Longitude	Transit Time to Next
S1	St. James	37.3397	-121.8896	3 min
S2	San Fernando	37.3365	-121.8901	3 min
S3	Convention Center	37.3310	-121.8893	3 min
S4	San Carlos	37.3290	-121.8900	3 min
S5	Tamien	37.3118	-121.8832	3 min
S6	Curtner	37.3020	-121.8750	3 min
S7	Capitol	37.2880	-121.8565	3 min
S8	Branham	37.2718	-121.8440	(terminus)

2.3 Output

- A ranked list of feasible apartments ordered by total weekly commute time (ascending).
- The shortest path from each apartment to each destination (showing intermediate stations).
- Performance comparison between Dijkstra and Bellman-Ford.

2.4 Constraints

- Monthly rent <= $1,700
- Walking time to nearest transit station <= 15 minutes

2.5 Objective Function

- For each feasible apartment a, the total weekly commute time is:
- TotalCommute(a) = sum over all destinations d_i of [shortest_time(a, d_i) x visits_per_week(d_i)]
- where shortest_time(a, d_i) is the minimum travel time through the VTA transit graph from apartment a to destination d_i, computed via Dijkstra's algorithm. The apartment with the lowest TotalCommute is the optimal choice.

3. Algorithm Design

3.1 Approach

We model the problem as a shortest-path problem on a weighted undirected graph. Each apartment, destination, and VTA Light Rail station is a node. Edges represent traversable routes with non-negative time-based weights (in minutes). We apply Dijkstra's algorithm with a min-heap priority queue to compute shortest paths from each apartment to all destinations.

Dijkstra is chosen over Bellman-Ford because all edge weights (walking times, riding times) are non-negative, allowing the more efficient O((V + E) log V) complexity versus Bellman-Ford's O(V * E). The overall pipeline is:

Build Graph -> Filter by Constraints -> Run Dijkstra per Apartment -> Compute Weighted Score -> Rank & Output

3.2 Graph Structure

The graph contains 16 nodes and 25 edges:

- **Node types:** 5 apartment nodes (A1-A5), 3 destination nodes (D1-D3), 8 station nodes (S1-S8).
- **Transit edges:** 7 edges connecting adjacent Light Rail stations (weight = 3 min each).
- **Walking edges:** Connect each apartment/destination to nearby stations within 15-minute walking distance. Weight = Haversine distance * 1.3 / walking speed (5 km/h).
- **Direct walking edges**: If an apartment and destination are within 30-minute walking distance, a direct edge is added so Dijkstra can compare walking vs. transit.

3.3 Pseudocode

BUILD_GRAPH:

```
function BUILD_GRAPH(apartments, destinations, stations):
    G = empty adjacency list
    for each pair of adjacent stations (s_i, s_{i+1}):
        add edge (s_i, s_{i+1}, weight = 3 min) to G     // bidirectional
    for each apartment a:
        for each station s within 15-min walking distance:
            t = haversine_distance(a, s) * 1.3 / walk_speed
            add edge (a, s, weight = t) to G
    for each destination d:
        (same as apartment edges above)
    for each (apartment a, destination d) within 30-min walk:
        add direct walking edge (a, d, weight = walk_time)
    return G
```

DIJKSTRA (with min-heap):

```
function DIJKSTRA(G, source):
```

```
    dist = {node: infinity  for all nodes in G}
    prev = {node: null      for all nodes in G}
    dist[source] = 0
    heap = min-heap containing (0, source)
    visited = empty set
    while heap is not empty:
        (d, u) = extract-min from heap
        if u in visited: continue
        add u to visited
        for each neighbor v of u with edge weight w:
            if v in visited: continue
            if d + w < dist[v]:
                dist[v] = d + w
                prev[v] = u
                insert (dist[v], v) into heap
    return dist, prev
```

FIND_BEST_APARTMENT:

```
function FIND_BEST(G, apartments, destinations, budget, max_walk):
    results = []
    for each apartment a:
        if a.rent > budget: skip
        if walk_to_nearest_station(a) > max_walk: skip
        dist, prev = DIJKSTRA(G, source = a)
        score = 0
        for each destination d:
            score += dist[d] * d.weekly_visits
        append (a, score, paths) to results
    sort results by score ascending
    return results
```

3.4 Time and Space Complexity

Let V = total number of nodes, E = total number of edges, A = number of candidate apartments, D = number of destinations.

Component	Time Complexity	Space Complexity
Dijkstra (single source)	O((V + E) log V)	O(V)
Full pipeline (A apartments)	O(A * (V + E) log V)	O(V + E) + O(A * D)
Bellman-Ford (comparison)	O(V * E)	O(V)

For our graph (V = 16, E = 25, A = 3 feasible apartments), the algorithm runs in under 0.1 milliseconds. Dijkstra is preferred over Bellman-Ford because all edge weights are non-negative (travel times cannot be negative).

4. Work Completed

4.1 Input Data (Real Locations)

All input data uses real San Jose addresses and coordinates. Five apartments were selected along the VTA Blue Line at varying distances from the NEU campus (4 N 2nd St, Downtown San Jose). Three destinations were chosen to represent typical student weekly errands in different directions. Eight VTA Light Rail stations were manually curated from the Blue Line segment. Coordinates were obtained from Google Maps. See the tables in Section 2 for the complete dataset.

4.2 Implementation Results

The optimizer has produced a complete ranked output. With a budget constraint of $1,700/month, apartments A1 ($2,200) and A2 ($1,800) were filtered out. The three feasible apartments were ranked as follows:

Rank	Apartment	Rent	Weekly Commute	Best Path to NEU
#1	A3 (Tamien)	$1,500	126.0 min	A3 -> S5 -> S4 -> S3 -> S2 -> D1
#2	A4 (Capitol)	$1,400	139.0 min	A4 -> S7 -> S6 -> S5 -> S4 -> S3 -> S2 -> D1
#3	A5 (Branham)	$1,350	149.2 min	A5 -> S8 -> S7 -> S6 -> S5 -> S4 -> S3 -> S2 -> D1

Interpretation

- A3 (Tamien) ranks #1 because Tamien station is centrally located on the Blue Line, providing efficient access to NEU (north) and the gym/grocery (south).
- A5 (Branham) has the lowest rent but the highest commute because it is furthest from NEU, which has the highest visit frequency (5x/week). However, Dijkstra found that A5 should walk directly to Planet Fitness (4.3 min) rather than taking transit (which would be slower). This demonstrates the algorithm correctly comparing walking vs. transit.
- The commute difference between #1 and #3 is 23 minutes/week (about 20 hours per year), which is significant for a student.

4.3 Implementation Progress

All core source code is complete and functional:

File	Lines	Description
utils.py	~50	Haversine distance, walking time estimation
graph.py	~100	Loads JSON data, builds weighted adjacency list graph

dijkstra.py	~100	Dijkstra with min-heap + Bellman-Ford + path reconstruction
optimizer.py	~200	Main pipeline: filter, score, rank, format output

Full source code is attached in the Appendix and available on GitHub.

5. Work in Progress

Task	Owner	Status	Est. Completion
Finalize data files with verified coordinates	Jiaxin Liu	90%	End of Week 11
Refine distance data	Jiaxin Liu	Not started	Week 12
Generate visualization of graph and paths	Fanchao Yu	Not started	Week 12
Algorithm improvement based on finalized data	Xujing Hui	Not started	Week 12
Draft final report and presentation slides	Fanchao Yu	In progress	Week 13

6. Planned Work / Next Steps

Task	Owner	Target	Depends On
Expand to apartments scope for richer analysis	Jiaxin Liu	Week 12	Data verified
Increase destinations from 3 fixed to multi-candidate	Jiaxin Liu	Week 12	Data verified
Refine weights	Jiaxin Liu	Week 12	Data verified
Create graph visualization (map overlay)	Fanchao Yu	Week 12	Data verified
Multi-modal routing in single graph	Xujing Hui	Week 12	Graph finalized
Improve Dijkstra's algorithm on multi-candidate selection	Xujing Hui	Week 12	Graph finalized
Write final report with analysis and figures	Fanchao Yu	Week 13	Results done
Prepare 15-minute presentation	All	Week 13	Report done

7. Issues, Challenges & Risks

Issue	Impact	Mitigation
Walking time estimation is approximate (Haversine * 1.3)	Results are estimates, not exact	Documented as a known limitation. Verified against Google Maps for sanity check.

We only selected 3 fixed destinations. Each apartment manually pre-assigned to a specific store. However, Dijkstra was not actually selecting the optimal destination.	The algorithm's core value, finding minimum-cost paths among multiple options, was bypassed. Results depended on human pre-selection rather than graph search	Expand to more destination nodes: 1 fixed (NEU) + 4 categories × 6 real candidate locations each (Costco, Trader Joe's, Starbucks, 24 Hour Fitness). No manual pre-assignment required.
The transit graph used only the VTA Blue Line. 8 stations in a linear chain with uniform 3-minute edge weights	Algorithm had no branching decisions to make and provided no routing insight beyond a fixed linear path	Rebuilt as a multi-modal network covering Caltrain, VTA bus hubs, Milpitas BART, and individual VTA bus stops across 8 routes, enabling meaningful shortest-path computation with real branching choices.
All 5 apartments were located along a single transit corridor, and the geographic diversity was minimal	Commute score differences between apartments were small and driven almost entirely by distance along one axis. The optimizer's ability to differentiate multi-directional access patterns could not be demonstrated	Apartments to span to more geographically distinct areas across three cities. Each apartment connects to the network through a different nearest station and route combination, ensuring that Dijkstra produces meaningfully different shortest paths and scores for each candidate.

8. Schedule Status

Status: Ahead of Schedule

We have completed more than originally planned for this reporting period. The working prototype with ranked results is operational, which was originally targeted for Week 9.

Milestone	Planned	Actual	Status
Problem formulation	Week 9	Week 9	Done
Algorithm design + pseudocode	Week 11	Week 13	Updating
Repository initialized	Week 10	Week 13	Updating
Data collection (5 apartments, 3 destinations)	Week 10	Week 12	Updating
Core implementation (graph + Dijkstra)	Week 12	Week 13	Done with most, still updating
Ranked results	Week 11	Week 13	Done with most, still updating

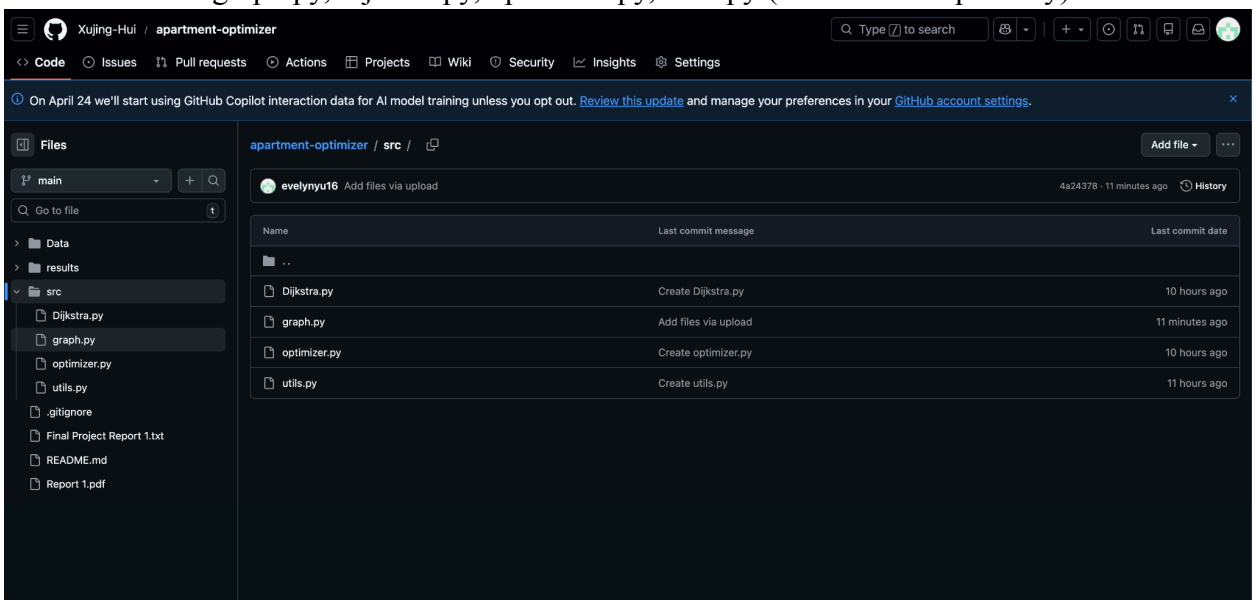
Performance comparison (Dijkstra vs B-F)	Week 9	Week 10	Done (ahead)
15 test cases passing	Week 11	Week13	Upcoming
Expand dataset + visualization	Week 10	Week12	Done with most, still updating
Final report + presentation	Week 11	Week 13	Upcoming

9. Conclusion

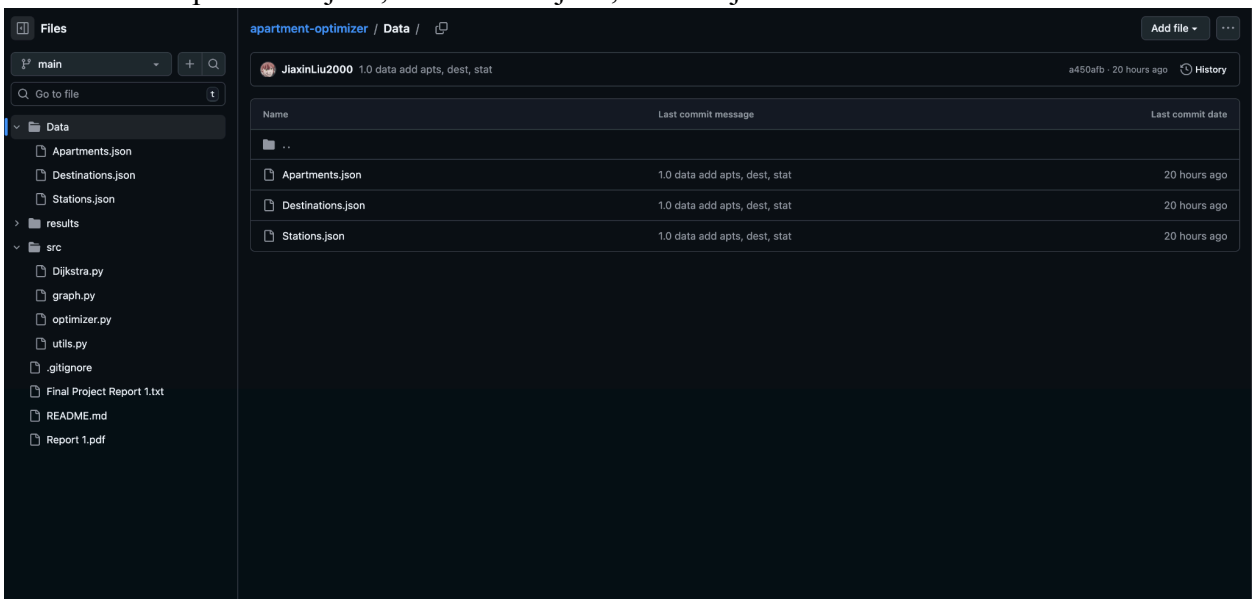
The project is progressing ahead of schedule. We have a fully operational prototype that correctly models the apartment selection problem as a graph shortest path problem, implements Dijkstra with a min heap, and produces meaningful ranked results using real San Jose apartments as examples. The algorithm correctly identifies that transit is not always faster than walking (as demonstrated by A5's direct walk to Planet Fitness), validating our graph design. Our next focus is expanding the dataset and preparing the final deliverables, especially with tests. We are confident in meeting all remaining deadlines.

10. Attachments

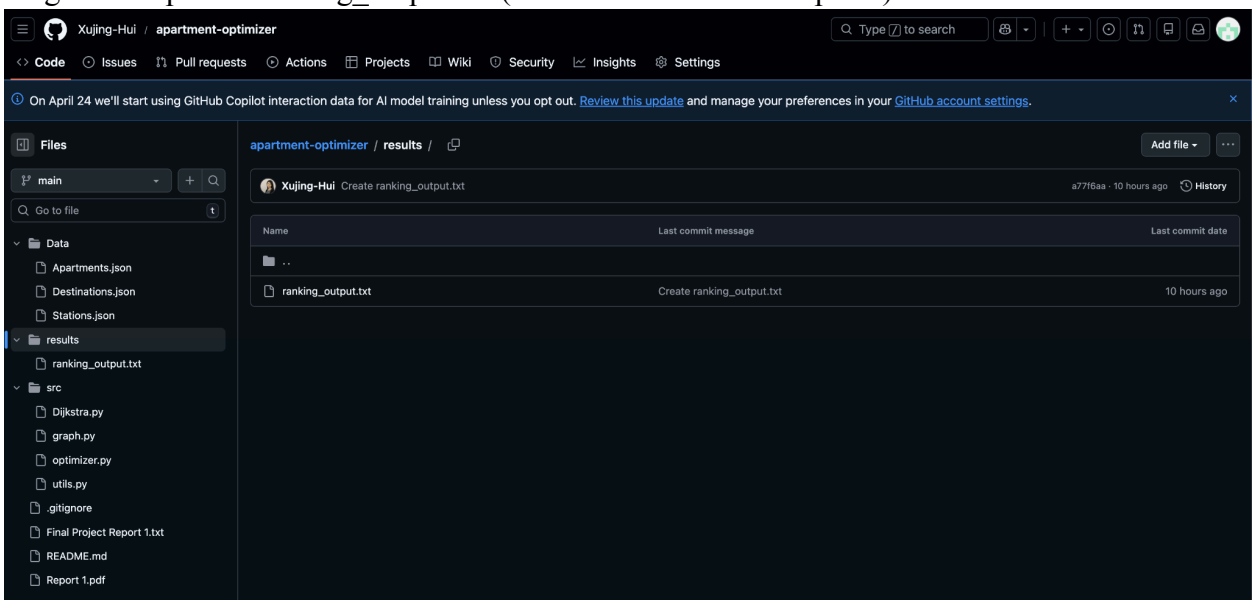
1. Source Code — graph.py, dijkstra.py, optimizer.py, utils.py (see GitHub repository)



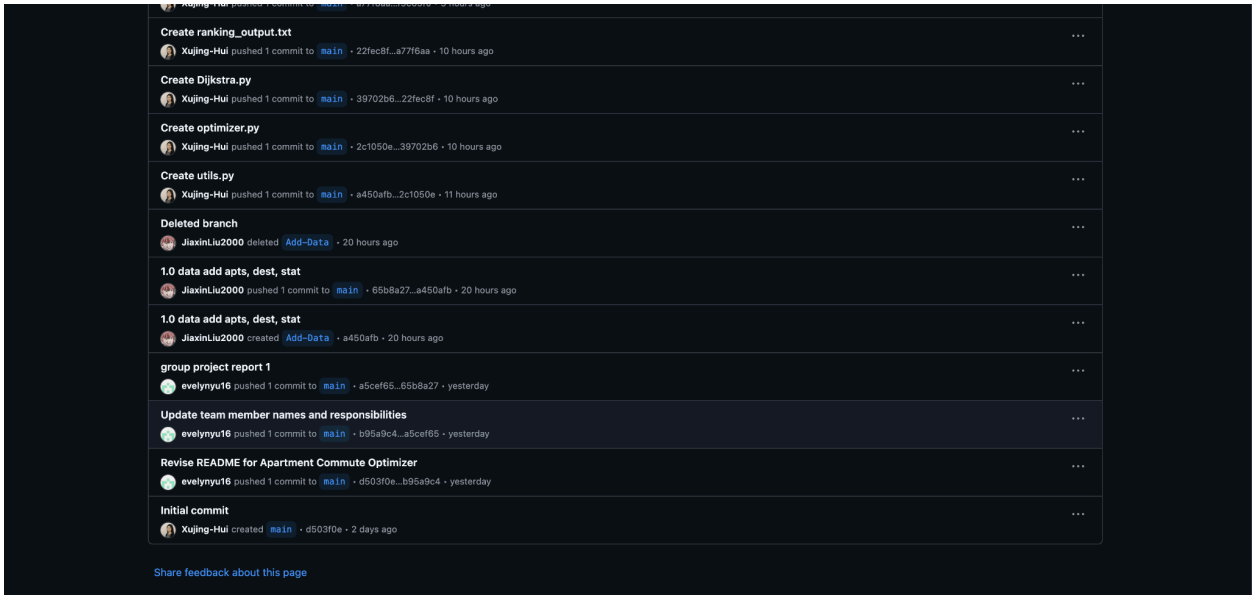
2. Data Files — apartments.json, destinations.json, stations.json



3. Program Output — ranking_output.txt (full ranked results with paths)



4. GitHub Push History — Screenshot showing commits from all 3 members



5. GitHub Repository: https://github.com/Xujing-Hui/apartment-optimizer